Title: Multi-scale method for ill-conditioned linear systems.

Name: Lazizbek Sadullaev

1. Introduction:

Solving linear systems of equations is a fundamental problem in scientific computing, engineering & applied mathematics. The problem can be formally stated as finding a vector x that satisfies the linear equation in a matrix form:

$$Ax = b \tag{1}$$

where

$A = [a_{ij}] \in \mathbb{R}^{n \times n}$ is non singular,

$x = [x_j] \in \mathbb{R}^n$ unknown, &

$b \in \mathbb{R}^n$ is a constant vector.

There are 2 approaches for solving (1):

(i) direct methods (LU decomposition)

(ii) iterative methods (Jacobi, Gauss-Siedel, SOR, GMRES, ...)

For large sparse (contains a lot of zero entries) systems, iterative methods are popular & efficient.

However, it's difficult to develop effective methods for ill-conditioned linear systems whose coefficient matrices have large condition numbers:

$$\mathrm{cond}_{\boxed{\cdot}}(A) = \|A\|_{\boxed{\cdot}} \cdot \|A^{-1}\|_{\boxed{\cdot}}$$

(We usually consider 2-norm unless specified.)

If the cond(A) is so large, the solution of the system (1) is so sensitive to small changes in the input data. We call this type of sytems ill-conditioned & this project explores a way of improving condition number, cond(A).

2. Multi-scale method:

If the matrix A of equations (1) is ill-conditioned the system (1) can be solved by the following equations:

$$(2) \qquad \tilde{A}\,\tilde{x} = b \qquad \text{where}$$

$$\tilde{A} = \begin{bmatrix} \frac{1}{\|\alpha_1\|_2} a_{11} & \frac{1}{\|\alpha_2\|_2} a_{12} & \cdots & \frac{1}{\|\alpha_n\|_2} a_{1n} \\ \frac{1}{\|\alpha_1\|_2} a_{21} & \frac{1}{\|\alpha_2\|_2} a_{22} & \cdots & \frac{1}{\|\alpha_n\|_2} a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{\|\alpha_1\|_2} a_{n1} & \frac{1}{\|\alpha_2\|_2} a_{n2} & \cdots & \frac{1}{\|\alpha_n\|_2} a_{nn} \end{bmatrix}; \quad \tilde{x} = \begin{bmatrix} \|\alpha_1\|_2 x_1 \\ \|\alpha_2\|_2 x_2 \\ \vdots \\ \|\alpha_n\|_2 x_n \end{bmatrix}$$

Here α_j, $j = 1, 2, \ldots n$, is the jth column of A.

with defined (2), the following multi-scale method is obtained for sytem (1):

The Multi-scale method:

Step 1: Solve $\tilde{A}\tilde{x} = b$ with iterative methods.

Step 2:

$$(3) \qquad x = \begin{bmatrix} \frac{1}{\|\alpha_1\|_2}\tilde{x}_1 \\ \frac{1}{\|\alpha_2\|_2}\tilde{x}_2 \\ \vdots \\ \frac{1}{\|\alpha_n\|_2}\tilde{x}_n \end{bmatrix} = \underbrace{\begin{bmatrix} \frac{1}{\|\alpha_1\|_2} & 0 & \cdots & 0 \\ 0 & \frac{1}{\|\alpha_2\|_2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \\ 0 & 0 & \cdots & \frac{1}{\|\alpha_n\|_2} \end{bmatrix}}_{D} \cdot \underbrace{\begin{bmatrix} \tilde{x}_1 \\ \tilde{x}_2 \\ \vdots \\ \tilde{x}_n \end{bmatrix}}_{\tilde{x}}$$

By this multi-scale method, we could improve the condition number of the coefficient matrix A.

i.e. (to be proved later)

$$(4) \qquad \text{cond}(\tilde{A}) \leq \text{cond}(A).$$

3. Application of the Method:

Consider the following sytem ($x_{exact} = [1, 1, 1]^T$ btw)

$$(5) \quad \underbrace{\begin{bmatrix} 1 & 1 & 0 \\ 0 & 100 & 1 \\ 1 & 0 & 10{,}000 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} 2 \\ 101 \\ 10{,}001 \end{bmatrix}}_{b}$$

The coefficient matrix A is almost singular (i.e. $\det(A) \to 0$). The condition number of A, we calculate below:

$$(6) \quad \text{cond}_2(A) = \|A\|_2 \cdot \|A^{-1}\|_2$$

We're considering 2-norm (spectral radius norm) as usual. For this we need to follow the steps below:

1). Compute the largest singular value $\sigma_{max}(A)$
2). Compute the smallest singular value $\sigma_{min}(A)$
3). $\text{cond}(A) = \dfrac{\sigma_{max}(A)}{\sigma_{min}(A)}$

The singular values of a matrix A are the square roots of the eigenvalues of $A^T A$. After I computed

$$(7) \quad \sigma(A^T A) \approx \{1;\ 10{,}000;\ 100000001\}$$

So the singular values are

$$(8) \quad \begin{cases} \sigma_1 = \sqrt{\lambda_1} \approx 1 \\ \sigma_1 = \sqrt{\lambda_2} \approx \sqrt{10,000} = 100 \\ \sigma_3 = \sqrt{\lambda_3} = \sqrt{100000001} = 10,000 \end{cases}$$

So the condition number of A

$$(9) \quad \text{cond}_2(A) = \frac{\sigma_{max}(A)}{\sigma_{min}(A)} = \frac{\sigma_3}{\sigma_1} \approx \frac{10,000}{1} = 10^4.$$

$\text{cond}_2(A) = 10^4$, high condition number indicates that the matrix A is ill-conditioned, meaning that iterative methods might not work well directly on sytem (5). This problem offers the use of multi-scale method.

To use multiscale method, let's transform our equation (5) to

$$(10) \quad \tilde{A}\,\tilde{x} = b \qquad \text{where}$$

$$\tilde{A} = \left[\frac{1}{\|\alpha_1\|_2} \cdot \alpha_1 \quad \frac{1}{\|\alpha_2\|_2} \alpha_2 \quad \frac{1}{\|\alpha_3\|_2} \alpha_3 \right]$$

where α_j, $j = 1, 2, 3$, is the jth column of A in (5).

So

$$(11)\quad \begin{cases} \|\alpha_1\|_2 = \|A[:,0]\|_2 = \sqrt{1^2+0^2+1^2} = \sqrt{2} \\ \|\alpha_2\|_2 = \|A[:,1]\|_2 = \sqrt{1^2+100^2+0^2} = \sqrt{10{,}001} \\ \|\alpha_3\|_2 = \|A[:,2]\|_2 = \sqrt{0^2+1^2+(10{,}000)^2} = \sqrt{100000001} \end{cases}$$

And

$$\tilde{A} = \left[\frac{1}{\sqrt{2}}\begin{bmatrix}1\\0\\1\end{bmatrix} \quad \frac{1}{\sqrt{10{,}001}}\begin{bmatrix}1\\100\\0\end{bmatrix} \quad \frac{1}{\sqrt{100000001}}\cdot\begin{bmatrix}0\\1\\10{,}000\end{bmatrix} \right] =$$

$$(12)\quad = \underbrace{\begin{bmatrix}1 & 1 & 0\\0 & 100 & 1\\1 & 0 & 10{,}000\end{bmatrix}}_{A} \cdot \underbrace{\begin{bmatrix}\frac{1}{\sqrt{2}} & 0 & 0\\0 & \frac{1}{\sqrt{10{,}001}} & 0\\0 & 0 & \frac{1}{\sqrt{100000001}}\end{bmatrix}}_{D} = A \cdot D .$$

for later reference A is the original coefficient matrix A, & D is the diagonal matrix of 2-norms of the columns of A.

In the next steps, we solve the new obtained system in (10) using an iterative method, in our project we focuse on Gauss-Seidel, to find $\tilde{x}$. Once we find $\tilde{x}$, we recover x by scaling back by D:

$$(13)\quad x = D\tilde{x} \qquad \text{by (3).}$$

Let me go through these steps now by expanding (5) $\tilde{A}\tilde{x} = b$ as follows:

$$(14) \quad \left[\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \quad \frac{1}{\sqrt{10{,}001}} \begin{bmatrix} 1 \\ 100 \\ 0 \end{bmatrix} \quad \frac{1}{\sqrt{100000001}} \begin{bmatrix} 0 \\ 1 \\ 10000 \end{bmatrix} \right] \begin{bmatrix} \tilde{x}_1 \\ \tilde{x}_2 \\ \tilde{x}_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 101 \\ 10{,}001 \end{bmatrix}$$

By Gauss-Seidel iteration formula in a matrix form

$$(15) \quad x^{(k+1)} = \left[-(D+L)^{-1} \cdot U \right] x^{(k)} + (D+L)^{-1} b .$$

In (15), if we let the starting iteration point $x^{(0)} = [0,0,0]^T$, and using the software to solve, the results in 5 steps with the tolerance $\varepsilon = 10^{-10}$ are as follows:

$$(16) \quad \tilde{x}^{(5)} = \begin{bmatrix} 0.7071\ldots \\ 0.0099\ldots \\ 9.9999\ldots \end{bmatrix}$$

So applying (13) to recover x:

$$(17) \quad x^{(5)} = D\tilde{x}^{(5)} = \begin{bmatrix} \frac{1}{\sqrt{2}} & 0 & 0 \\ 0 & \frac{1}{\sqrt{10000}} & 0 \\ 0 & 0 & \frac{1}{\sqrt{100000001}} \end{bmatrix} \cdot \begin{bmatrix} 0.7071\ldots \\ 0.0099\ldots \\ 9.9999\ldots \end{bmatrix} \approx \begin{bmatrix} 0.999\ldots \\ 1.0 \\ 1.0 \end{bmatrix}$$

And the relative error for the multi-scale method in <u>5 steps</u> is

$$(18) \quad e_{ms} = \| x^{(5)} - x_{exact} \|_2 = \left\| \begin{bmatrix} 0.999 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right\|_2 = 9.769963 \cdot 10^{-15}$$

To verify the reasoning behind the multiscale method, we can compare the condition numbers of A & $\tilde{A}$.

(19) $$\underbrace{\mathrm{cond}_2(A) = 10^4}_{\text{(calculated earlier)}} > \underbrace{2.4143808 = \mathrm{cond}_2(\tilde{A})}_{\text{(calculated in a similar way, software code is in appendix)}}$$

that is matching our formula in (4):

(4) $$\mathrm{cond}(A) \geq \mathrm{cond}(\tilde{A})$$

4. Remarks for the Multi-scale method (specifically for (4) above):

(20) $$\tilde{A} = \begin{bmatrix} \frac{a_{11}}{\|\alpha_1\|_2} & \cdots & \frac{a_{1n}}{\|\alpha_n\|_2} \\ \vdots & \ddots & \vdots \\ \frac{a_{n1}}{\|\alpha_1\|_2} & \cdots & \frac{a_{nn}}{\|\alpha_n\|_2} \end{bmatrix} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \\ a_{n1} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} \frac{1}{\|\alpha_1\|_2} & 0 & \cdots & 0 \\ 0 & \frac{1}{\|\alpha_2\|_2} & & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{\|\alpha_n\|_2} \end{bmatrix} = AD$$

Thus, for simplicity, if we let the condition number to be denoted by 1-norm:

(21) $$d_{max} = \max\{\|\alpha_1\|_2, \|\alpha_2\|_2, \ldots, \|\alpha_n\|_2\}$$

(22) $$d_{min} = \min\{\|\alpha_1\|_2, \|\alpha_2\|_2, \ldots, \|\alpha_n\|_2\}$$

the following holds

property $\|AB\| \leq \|A\| \cdot \|B\|$

$$\mathrm{cond}_1(\tilde{A}) = \|\tilde{A}\|_1 \cdot \|\tilde{A}^{-1}\|_1 = \|AD\|_1 \cdot \|(AD)^{-1}\|_1 = \|A \cdot D\|_1 \cdot \|D^{-1} \cdot A^{-1}\|_1 \leq \|A\|_1 \|D\|_1 \cdot \|D^{-1}\|_1 \cdot \|A^{-1}\|_1$$

$$= \|A\|_1 \cdot \frac{d_{max}}{d_{min}} \cdot \|A^{-1}\|_1 = \frac{d_{max}}{d_{min}} \|A\|_1 \cdot \|A^{-1}\|_1 = \frac{d_{max}}{d_{min}} \cdot \mathrm{cond}(A);$$

That is

$$(23) \qquad \mathrm{cond}(\tilde{A}) \leq \frac{d_{max}}{d_{min}} \mathrm{cond}(A),$$

therefore the condition number of $\tilde{A}$, $\mathrm{cond}(\tilde{A})$ could be no more than that of A, $\mathrm{cond}(A)$.

5. Conclusion:

This project discussed to solve system of linear equations (1) which is ill-conditioned. By carefully scaling the matrix and applying iterative methods, we can achieve better numerical stability and accuracy.

6. Comparisons & appendix:

To compare the results as tables obtained by different iterative methods (namely Gauss-Siedel, Jacobi), and our defined multiscale method, I'll provide a software code for all methods and calculations used in the appendix below as a further reference.

Final_Project_Math548_Lazizbek

June 28, 2024

1 Lazizbek Sadullaev

2 Final Project, Math 548

#Example problems to compare the methods:

Solve the following systems of equations first using the Jacobi iteration method, then Gauss-Seidel iteration method, and finally new introduced Multi-scale method. In each case, compare the numerical results.

1. [1 1 0][x1] [2]

   [0 100 1][x2] = [101]

   [1 0 10000][x3] [10001]

2. [1 (1/2) (1/3)][x1] [1]

   [(1/2) (1/3) (1/4)][x2] = [1]

   [(1/3) (1/4) (1/5)][x3] [1]

3 Solving Ax=b matrix equation:

Source:

[ ]:
```
# To solve Ax=b, We start by constructing the arrays for A and b.

import numpy as np
A = np.array([[ 1,  1,  0],
              [-2,  0,  2],
              [ 4, -4,  0]])
b = np.transpose(np.array([ (-7/16),  (11/8),  0]))
# To solve the system we do

x = np.linalg.solve(A,b)
print("Real solution: ")
print(x)
```

4 Finding eigenstuff of a matrix:

Source:

[ ]:
```
# To find the eigenvalues and eigenvectors we do
import numpy as np
from numpy.linalg import eig
a = np.array([[0, 2],
              [2, 3]])
w,v=eig(a)
print('E-value:', w)
print('E-vector', v)
```

```
E-value: [-1.  4.]
E-vector [[-0.89442719 -0.4472136 ]
 [ 0.4472136  -0.89442719]]
```

5 Example 1. Finding exact solution:

[ ]:
```
# To solve Ax=b, We start by constructing the arrays for A and b.

import numpy as np
A = np.array([[ 1,  1,  0],
              [ 0,  100,  1],
              [ 1,  0,  10000]])
b = np.transpose(np.array([ 2, 101, 10001]))
# To solve the system we do

x = np.linalg.solve(A,b)
print("Real solution: ")
print(x)
```

```
Real solution:
[1. 1. 1.]
```

6 Solving Example 1 using Jacobi, Gauss-Siedel, and Multi-scale methods for comparison:

[ ]:
```
import numpy as np
import pandas as pd
from scipy.sparse.linalg import LinearOperator, gmres

# Define the coefficient matrix A and the right-hand side vector b
A = np.array([
    [1, 1, 0],
    [0, 100, 1],
```

```
    [1, 0, 10000]
])

b = np.array([2, 101, 10001])

# Define the exact solution for comparison
x_exact = np.linalg.solve(A, b)

# Define a function for the Jacobi method
def jacobi(A, b, x0, tol=1e-10, max_iterations=1000):
    n = len(b)
    x = x0.copy()
    D = np.diag(A)
    R = A - np.diagflat(D)
    for i in range(max_iterations):
        x_new = (b - np.dot(R, x)) / D
        if np.linalg.norm(x_new - x, ord=np.inf) < tol:
            break
        x = x_new
    return x, i+1

# Define a function for the Gauss-Seidel method
def gauss_seidel(A, b, x0=None, tol=1e-10, max_iterations=1000):
    n = len(b)
    if x0 is None:
        x0 = np.zeros(n)

    x = x0.copy()
    D = np.diag(np.diag(A))
    L = np.tril(A, -1)
    U = np.triu(A, 1)

    for k in range(max_iterations):
        x_new = np.dot(np.linalg.inv(D + L), b - np.dot(U, x))

        if np.linalg.norm(x_new - x, ord=np.inf) < tol:
            return x_new, k
        x = x_new

    return x, max_iterations


# Define a function for the Multi-scale method using the Gauss-Seidel, Jacobi,␣
↪GMRES methods
def multi_scale(A, b, tol=1e-10, max_iterations=1000):
    D = np.linalg.norm(A, axis=0)  # Compute the Diagonal Scaling Matrix D
    D = np.diag(D)
```

```
    A_tilde = np.dot(A, D)  # Scale the Matrix A
    # print(A_tilde)
    b_tilde = b  # b does not change
    x0 = np.zeros_like(b)  # Initialize the Starting Guess

    # Solve the scaled system using Gauss-Seidel
    x_tilde, num_iterations = gauss_seidel(A_tilde, b_tilde, x0=x0, tol=tol,␣
↪max_iterations=max_iterations)

    # Solve the scaled system using Jacobi
    # x_tilde, num_iterations = jacobi(A_tilde, b_tilde, x0=x0, tol=tol,␣
↪max_iterations=max_iterations)

    # Solve the scaled system using GMRES(Generalized Minimal Residual) by␣
↪defined LinearOperator
    # def mv(v):
    #     return np.dot(A_tilde, v)

    # A_op = LinearOperator(A_tilde.shape, matvec=mv)
    # x_tilde, num_iterations = gmres(A_op, b_tilde, x0=x0, tol=tol,␣
↪maxiter=max_iterations)

    # Scale the solution back to the original system
    x = np.dot(D, x_tilde)
    # x = x_tilde

    return x, num_iterations

# Initial guess for the iterative methods
x0 = np.zeros_like(b)

# Solve using the Jacobi method
x_jacobi, it_jacobi = jacobi(A, b, x0)
err_jacobi = np.linalg.norm(x_jacobi - x_exact)

# Solve using the Gauss-Seidel method
x_gs, it_gs = gauss_seidel(A, b, x0)
err_gs = np.linalg.norm(x_gs - x_exact)

# Solve using the Multi-scale method
x_ms, it_ms = multi_scale(A, b)
err_ms = np.linalg.norm(x_ms - x_exact)

# Create a DataFrame to display the results
results = pd.DataFrame({
    'Method': ['Jacobi', 'Gauss-Seidel', 'Multi-scale'],
    'Iterations (IT)': [it_jacobi, it_gs, it_ms],
```

```
    'Relative Error (ERR)': [err_jacobi, err_gs, err_ms],
    'Final Solution': [x_jacobi, x_gs, x_ms]
})

print(results)
```

```
         Method  Iterations (IT)  Relative Error (ERR)  \
0        Jacobi                7          1.732141e-12
1  Gauss-Seidel                5          9.992007e-15
2   Multi-scale                5          9.769963e-15

                                      Final Solution
0  [0.9999999999989999, 0.999999999999, 0.9999999…
1                       [0.99999999999999, 1.0, 1.0]
2                     [0.9999999999999902, 1.0, 1.0]
```

7 Computing the singular values of matrices in Example 1:

[ ]:
```
import numpy as np

# Define the original matrix A
A = np.array([
    [1, 1, 0],
    [0, 100, 1],
    [1, 0, 10000]
])

# Compute the singular values of the original matrix A
singular_values_A = np.linalg.svd(A, compute_uv=False)

# Compute the condition number of the original matrix A
cond_A = max(singular_values_A) / min(singular_values_A)

# Print the results
print(f"The original matrix A is:\n{A}")
print("\nThe singular values of A are: ", singular_values_A)
print(f"The condition number of A is: {cond_A}")
```

```
The original matrix A is:
[[    1     1     0]
 [    0   100     1]
 [    1     0 10000]]

The singular values of A are:  [1.00000001e+04 1.00005000e+02 9.99950994e-01]
The condition number of A is: 10000.490186030387
```

[ ]:
```python
import numpy as np
# Define the original matrix A
A = np.array([
[1, 1, 0],
[0, 100, 1],
[1, 0, 10000]
])
# Compute the diagonal scaling matrix D
D_values = np.linalg.norm(A, axis=0)
D = np.diag(1 / D_values)
# Compute the scaled matrix A_tilde
A_tilde = np.dot(A, D)
# Compute the singular values of the scaled matrix A_tilde
singular_values = np.linalg.svd(A_tilde, compute_uv=False)
# Compute the condition number
cond_A_tilde = max(singular_values) / min(singular_values)
print(f"The scaled matrix A_tilde is:\n{A_tilde}")
print("\nThe singular_values of A_tilde is: ", singular_values)
print(f"The condition number of A_tilde is: {cond_A_tilde}")
```

```
The scaled matrix A_tilde is:
[[7.07106781e-01 9.99950004e-03 0.00000000e+00]
 [0.00000000e+00 9.99950004e-01 9.99999995e-05]
 [7.07106781e-01 0.00000000e+00 9.99999995e-01]]

The singular_values of A_tilde is:  [1.30657688 0.999999   0.54116436]
The condition number of A_tilde is: 2.4143808690684616
```

8 Example 2. Finding exact solution:

[ ]:
```python
import numpy as np

# Define the matrix A
A = np.array([[1, 1/2, 1/3],
              [1/2, 1/3, 1/4],
              [1/3, 1/4, 1/5]])

# Define the vector b
b = np.array([1, 1, 1])

# Solve the linear system Ax = b
x = np.linalg.solve(A, b)

# Print the solution
print("Real solution: ")
print(x)
```

```
Real solution:
[  3. -24.  30.]
```

9 Solving Example 2 using Jacobi, Gauss-Siedel, and Multi-scale methods for comparison:

[ ]:
```python
import numpy as np
import pandas as pd
from scipy.sparse.linalg import LinearOperator, gmres

# Define the coefficient matrix A and the right-hand side vector b
A = np.array([[1, 1/2, 1/3],
              [1/2, 1/3, 1/4],
              [1/3, 1/4, 1/5]])

# Define the vector b
b = np.array([1, 1, 1])

# Define the exact solution for comparison
x_exact = np.linalg.solve(A, b)

# Define a function for the Jacobi method
def jacobi(A, b, x0, tol=1e-10, max_iterations=1000):
    n = len(b)
    x = x0.copy()
    D = np.diag(A)
    R = A - np.diagflat(D)
    for i in range(max_iterations):
        x_new = (b - np.dot(R, x)) / D
        if np.linalg.norm(x_new - x, ord=np.inf) < tol:
            break
        x = x_new
    return x, i+1

# Define a function for the Gauss-Seidel method
def gauss_seidel(A, b, x0=None, tol=1e-10, max_iterations=1000):
    n = len(b)
    if x0 is None:
        x0 = np.zeros(n)

    x = x0.copy()
    D = np.diag(np.diag(A))
    L = np.tril(A, -1)
    U = np.triu(A, 1)

    for k in range(max_iterations):
```

```
        x_new = np.dot(np.linalg.inv(D + L), b - np.dot(U, x))

        if np.linalg.norm(x_new - x, ord=np.inf) < tol:
            return x_new, k
        x = x_new

    return x, max_iterations


# Define a function for the Multi-scale method using the Gauss-Seidel, Jacobi,␣
↪GMRES methods
def multi_scale(A, b, tol=1e-10, max_iterations=1000):
    D = np.linalg.norm(A, axis=0)  # Compute the Diagonal Scaling Matrix D
    D = np.diag(D)
    A_tilde = np.dot(A, D)  # Scale the Matrix A
    # print(A_tilde)
    b_tilde = b  # b does not change
    x0 = np.zeros_like(b)  # Initialize the Starting Guess

    # Solve the scaled system using Gauss-Seidel
    x_tilde, num_iterations = gauss_seidel(A_tilde, b_tilde, x0=x0, tol=tol,␣
↪max_iterations=max_iterations)

    # Solve the scaled system using Jacobi
    # x_tilde, num_iterations = jacobi(A_tilde, b_tilde, x0=x0, tol=tol,␣
↪max_iterations=max_iterations)

    # Solve the scaled system using GMRES(Generalized Minimal Residual) by␣
↪defined LinearOperator
    # def mv(v):
    #     return np.dot(A_tilde, v)

    # A_op = LinearOperator(A_tilde.shape, matvec=mv)
    # x_tilde, num_iterations = gmres(A_op, b_tilde, x0=x0, tol=tol,␣
↪maxiter=max_iterations)

    # Scale the solution back to the original system
    x = np.dot(D, x_tilde)
    # x = x_tilde

    return x, num_iterations

# Initial guess for the iterative methods
x0 = np.zeros_like(b)

# Solve using the Jacobi method
x_jacobi, it_jacobi = jacobi(A, b, x0)
```

```
err_jacobi = np.linalg.norm(x_jacobi - x_exact)

# Solve using the Gauss-Seidel method
x_gs, it_gs = gauss_seidel(A, b, x0)
err_gs = np.linalg.norm(x_gs - x_exact)

# Solve using the Multi-scale method
x_ms, it_ms = multi_scale(A, b)
err_ms = np.linalg.norm(x_ms - x_exact)

# Create a DataFrame to display the results
results = pd.DataFrame({
    'Method': ['Jacobi', 'Gauss-Seidel', 'Multi-scale'],
    'Iterations (IT)': [it_jacobi, it_gs, it_ms],
    'Relative Error (ERR)': [err_jacobi, err_gs, err_ms],
    'Final Solution': [x_jacobi, x_gs, x_ms]
})

print(results)
```

```
         Method  Iterations (IT)  Relative Error (ERR)  \
0        Jacobi             1000                   inf
1  Gauss-Seidel             1000          1.650781e-07
2   Multi-scale             1000          1.650779e-07

                                      Final Solution
0  [-1.095738359843396e+236, -2.0686415929498956e…
1  [2.9999999764793808, -23.9999998799592, 29.999…
2  [2.9999999764793963, -23.999999879959315, 29.9…
```

10 Computing the singular values of matrices in Example 2:

[ ]:
```
import numpy as np

# Define the original matrix A
A = np.array([[1, 1/2, 1/3],
              [1/2, 1/3, 1/4],
              [1/3, 1/4, 1/5]])

# Compute the singular values of the original matrix A
singular_values_A = np.linalg.svd(A, compute_uv=False)

# Compute the condition number of the original matrix A
cond_A = max(singular_values_A) / min(singular_values_A)

# Print the results
```

```
print(f"The original matrix A is:\n{A}")
print("\nThe singular values of A are: ", singular_values_A)
print(f"The condition number of A is: {cond_A}")
```

```
The original matrix A is:
[[1.         0.5        0.33333333]
 [0.5        0.33333333 0.25      ]
 [0.33333333 0.25       0.2       ]]

The singular values of A are:  [1.40831893 0.12232707 0.00268734]
The condition number of A is: 524.0567775860644
```

[ ]:
```
import numpy as np
# Define the original matrix A
A = np.array([[1, 1/2, 1/3],
              [1/2, 1/3, 1/4],
              [1/3, 1/4, 1/5]])

# Compute the diagonal scaling matrix D
D_values = np.linalg.norm(A, axis=0)
D = np.diag(1 / D_values)
# Compute the scaled matrix A_tilde
A_tilde = np.dot(A, D)
# Compute the singular values of the scaled matrix A_tilde
singular_values = np.linalg.svd(A_tilde, compute_uv=False)
# Compute the condition number
cond_A_tilde = max(singular_values) / min(singular_values)
print(f"The scaled matrix A_tilde is:\n{A_tilde}")
print("\nThe singular_values of A_tilde is: ", singular_values)
print(f"The condition number of A_tilde is: {cond_A_tilde}")
```

```
The scaled matrix A_tilde is:
[[0.85714286 0.76822128 0.72121845]
 [0.42857143 0.51214752 0.54091383]
 [0.28571429 0.38411064 0.43273107]]

The singular_values of A_tilde is:  [1.72408534 0.16585673 0.0046133 ]
The condition number of A_tilde is: 373.72034839797374
```

11 To download

[ ]:
```
# !sudo apt-get install texlive-xetex texlive-fonts-recommended␣
↪texlive-plain-generic
```

[ ]:
```
# !jupyter nbconvert --to pdf /content/Math548_hw6_Lazizbek.ipynb
```